

END-TO-END DATA PROJECT

Do CRM/ERP à ação comercial no Databricks



Pipeline medalhão · Modelo de score de contato ·
Genie Space · App de atendimento — construídos
com Databricks Asset Bundles e Claude Code via
MCP.

3.041	28.7k	293
CLIENTES	PEDIDOS	SKUS

► Navegue pelas páginas para conhecer o projeto

O PROBLEMA

Times comerciais decidindo no escuro

CRM e ERP viviam em silos. Ninguém sabia, de forma sistemática, quais clientes precisavam de contato agora — e por quê.



37.937

visitas registradas sem cruzamento
com risco de churn ou inadimplência



5.980

oportunidades no funil, muitas
paradas sem alerta ou follow-up



217

clientes já inativos sem nenhuma
ação de reativação estruturada

A pergunta que guiou o projeto:

"Quais clientes precisam de contato, e por quê?"

Databricks Asset Bundle

— como o projeto foi estruturado

```
projeto_databricks/  
├─ databricks.yml      # Bundle: targets dev/prod  
├─ resources/  
│   ├─ catalogo.yml    # Schemas bronze/silver/gold + Volume  
│   ├─ pipeline.job.yml # Job orquestrado (10 tasks em DAG)  
│   └─ dashboard-comercial.lvdash.json  
├─ src/  
│   ├─ raw/conferencia.py # Valida CSVs no Volume  
│   ├─ bronze/ingestao.py # 10 tabelas Delta (tudo STRING)  
│   ├─ silver/01..04.py   # Limpeza, tipos, constraints CHECK  
│   └─ gold/05..08.py     # Dims, Fato, Marts, 9 testes  
├─ scripts/criar-catalogo.sh  
└─ dados/  crm/ + erp/    # 10 CSVs de origem
```

targets dev/prod

Sem mode: development — evita prefixo de schema que quebra o SQL

DAG de 10 tasks

raw → bronze → 4x silver (paralelo) → gold dims → fato → marts → testes → dashboard

Serverless

Notebooks PySpark sem SparkContext — usa dbutils.fs para inspecionar volumes

Bronze → Silver → Gold

BRONZE 10 tabelas Delta — dado cru 100% STRING

- inferSchema=False: sem adivinhação de tipos
- Metadados técnicos: _ingerido_em, _path_origem
- Conferência: linhas da tabela == linhas do CSV (raise se divergir)
- Idempotente: overwrite a cada run

SILVER 10 tabelas tipadas com contratos de qualidade

- CNPJ normalizado a 14 dígitos (lpad + regexp_replace)
- Datas: try_to_date() — ANSI mode, nunca aborta
- Deduplicação por CNPJ (mantém cadastro mais antigo)
- Constraints CHECK: ALTER TABLE ADD CONSTRAINT (idempotente)

GOLD Dimensões, Fato, Marts e 9 testes de qualidade

- 4 dimensões: dim_cliente, dim_produto, dim_vendedor, dim_calendario
- fato_vendas particionada por ano/mes — exclui cancelados
- 3 marts por diretoria: Comercial, Produto, Financeiro
- 9 testes raise_error() — job para se qualquer falhar

Licao aprendida:

mode: development prefixa schemas (dev_user_bronze) e quebra todo o SQL.
Solucao: pause_status: PAUSED no job.

Star schema + 3 marts

prontos para BI e Genie



3 MARTS DE CONSUMO:

mart_vendas_ por_vendedor

Ticket médio,
meta, atingimento

mart_produto_ performance

Curva ABC,
rank mensal

mart_financeiro_ recebimento

Adimplência,
atraso médio

9 testes automáticos + contratos na Silver



Receita gold

= silver ($\pm 0,01$)
raise_error() se falhar



CNPJ único

na silver
raise_error() se falhar



data_pedido

sem nulos
raise_error() se falhar



Receita negativa

só com devolução
raise_error() se falhar



Volume fato_vendas

entre 140k-250k
raise_error() se falhar



pedido_id

sem órfão
raise_error() se falhar



cliente_id

sem órfão
raise_error() se falhar



Mart produto

$\equiv$ fato ($\pm 0,01$)
raise_error() se falhar



CNPJ

com 14 dígitos
raise_error() se falhar

Contratos na Silver (CHECK Constraints)

silver.clientes → cnpj_14_digitos, data_cadastro_existe
silver.pedidos → valor_total_positivo, data_pedido_existe
silver.produtos → custo_unitario_positivo, preco_tabela_positivo

Unity Catalog:

Lineage automático · COMMENT em tabelas e colunas · PII tag em CNPJ

Dashboard como código, versionado no bundle

O arquivo dashboard-comercial.lvdash.json é declarado no bundle (dashboard.dashboard.yml) e republicado a cada run do job — garantindo diff, rollback e re-deploy.

Receita Líquida

Bruto - devoluções.
Particionado ano/mes.

Margem %

receita - custo / receita.
Regra vive no fato.

Ticket Médio

Receita / pedidos.
1 pedido = 1 ticket.

Atingimento Meta

Receita / meta_mensal.
NULL se sem meta.

Queries sem prefixo

FROM fato_vendas — sem catalog.schema. Dashboard portavel.

Filtros cruzados

Canal x Vendedor x Período x Categoria de produto

Refresh automatico

task dashboard_refresh no final do DAG — sempre atualizado

Versionado em Git

.lvdash.json no repo — historico, PR, rollback

ENCERRAMENTO

O que fica desse projeto

- Medalhão + Unity Catalog dá confiança ao time de negócio — dado confiável é pré-requisito de ação.
- Dashboard como código (.lvdash.json no bundle) = diff, rollback e deploy em 1 comando.
- 9 testes automáticos com `raise_error()` fazem o job parar antes do dado ruim chegar ao BI.
- Agentic engineering (Claude Code + MCP + Databricks AI Dev Kit) acelerou todo o scaffold.

STACK

Databricks (DABs, Unity Catalog, Serverless, AI/BI Dashboard)
PySpark · Delta Lake · SQL ANSI · Claude Code + MCP

Código completo no GitHub

github.com/baludf/projeto_databricks

Comenta se quiser trocar ideia sobre agentic engineering no Databricks ↓

Alt: Slide de encerramento — aprendizados, stack e link do repositório GitHub.